

1 Temporal GRPO: 超越 Vision-Language-Action 强化学习中的轨迹级 credit 分配

2 摘要

结果驱动的强化学习为利用稀疏的任务成功反馈对 Vision-Language-Action (VLA) policy 进行后训练提供了一种可扩展的途径。在常见的基于 GRPO 的 VLA 后训练中, 轨迹级别的单个 advantage 会被施加到轨迹中的每一个动作上。因此, 一条 rollout 即使已经正确完成了若干有效阶段, 但若在后续阶段失败, 先前推动任务进展的动作仍可能受到惩罚。我们将这一现象称为**轨迹级 credit 混叠 (trajectory-level credit aliasing)**。

Temporal GRPO 通过构造可检测的任务阶段, 将每条 rollout 与阶段对应的动作区间对齐, 并且仅在进入同一阶段的 rollouts 之间进行比较, 从而缓解这一问题。由此得到的阶段 advantage 会在一次 policy 更新中施加到各自对应的动作区间上。

在 RoboTwin 2.0 上, Temporal GRPO 在任务成功率 (success rate) 和样本效率方面均优于基线方法, 并且在不同任务时域长度下都表现出稳定增益。在 LIBERO-Long 上, 受控更新实验表明, 该方法能够保留任务间共享的前置阶段, 并将改进集中在 rollout 结果首次出现分化的阶段。

3 Introduction

近年来, Vision-Language-Action (VLA) 模型通过对视觉观测、语言指令与机器人动作进行联合建模, 在多任务机器人操纵中展现出很强的潜力; 但由于 embodiment、场景分布和任务目标存在差异, 模型在实际应用中仍然需要面向下游任务进行适配 (Brohan et al. 2023; Kim et al. 2024; Li et al. 2024; NVIDIA et al. 2025)。为降低对额外专家 demonstration 的依赖, 近期工作开始采用强化学习对预训练 VLA 进行 post-training, 使 policy 能够通过环境交互与任务反馈持续改进 (Li et al. 2025; Chen et al. 2026; Lu et al. 2025; Zhang et al. 2025b; Chen et al. 2025c; Black et al. 2026)。具有代表性的、结果驱动的 VLA-RL 方法, 通常会针对同一任务采样多条完整轨迹, 根据它们最终的成功或失败计算 group-relative advantage, 再利用这些 advantage 更新所采样的动作序列。

然而, 一条最终失败的 rollout 可能已经正确完成了若干前置阶段, 只是在后续某个阶段才失败。例如, 早期失败与后期失败的 rollout, 尽管任务进展存在显著差异, 却可能获得相同的标量轨迹 advantage。当这种负 advantage 被广播到整段已执行的动作序列时, 那些推动有效阶段进展的前序行为, 会与真正导致失败的动作一起被压制。我们将这种“不同阶段结果被压缩为同一轨迹信号”的现象称为**轨迹级 credit aliasing**。它会使 policy 更新与局部任务进展失配, 并削弱长时程操纵中的样本效率与子技能稳定性。

这种 credit aliasing 的根源在于: 长时程任务的最终结果实际上聚合了多个阶段的结果, 因此, 具有不同任务进展模式的 rollout 可能被映射为同一个失败标签和同一个轨迹 advantage。于是, 若直接比较那些完成了不同前置阶段、并在不同时间点失败的 rollout, 得到的轨迹级 advantage 只反映总体结果差异, 而无法指出这种差异究竟从哪个阶段开始产生。我们观察到, 完成了相同先决阶段的 rollout 具有相同的任务进展, 并面临同一个当前阶段完成问题, 因此它们构成了一个更具可比性的局部分组。在这一共享条件下, 进入同一阶段但阶段结果不同的 rollout, 可以为该阶段提供有信息量的 group-relative 排序信号。据此, advantage 估计可以从“对完整轨迹结果进行无条件比较”, 重构为“在共享先决进展条件下进行阶段条件比较”; 并且, 每个阶段的 advantage 只分配给产生相

应阶段结果的动作区间。

基于上述分析，本文提出 Temporal GRPO，这是一种面向结果驱动 VLA post-training 的、阶段条件化时序 credit assignment 框架。给定任务指令和初始观测，Temporal GRPO 会构建一个可检测的、有序任务阶段序列，并将每条完整 rollout 与对应的阶段及动作区间对齐。对于每个阶段，该方法只比较那些满足共享先决条件且确实进入该阶段的 rollout，并根据它们在该阶段的完成结果计算 group-relative advantage。随后，将得到的阶段 advantage 分配给产生相应阶段结果的动作区间，从而在保持最终任务成功目标、完整 rollout 形式以及单一 VLA policy 不变的前提下，实现分段式的时序 credit 分配。本文的主要贡献如下：

- 我们刻画并形式化了结果驱动 VLA 强化学习中的轨迹级 credit aliasing：具有不同阶段进展的 rollout 可能获得相同的最终结果 advantage，从而使成功的前序动作与后续失败动作被同一 advantage 共同更新。
- 我们提出 Temporal GRPO。该方法构建可检测的有序阶段，按进入阶段对 rollout 进行分组，并仅将每个阶段的相对 advantage 分配给对应的动作区间。

4 引言（第二部分）

在训练预算一致的条件下，Temporal GRPO 在 RoboTwin 2.0 上取得了 75.8% 的宏平均成功率 (macro-average success rate)，相较于最强的受控 baseline 提升了 7.0 个百分点；同时，在所有任务时域上都带来了稳定增益，提升幅度为 6.2–8.3 个百分点。对受控 LIBERO-Long 更新以及消融实验的进一步分析表明，这些增益来源于两点：一是能够保留已习得的前序行为，二是将 policy 更新集中在那些真正导致结果差异的动作时间区间上。

4.1 Related Works

4.2 面向 VLA 的结果驱动强化学习

VLA 模型通常先从大规模机器人 demonstration 中获得通用操纵能力，再利用下游数据适配特定 embodiment 与任务；近期研究进一步采用强化学习后训练，通过与环境交互并利用任务反馈，弥补离线 demonstration 覆盖范围有限的问题 (Kim et al. 2024; Zhang et al. 2025a; Li et al. 2025; Tan et al. 2025)。在这类方法中，结果驱动方法直接利用完整任务的成功信号优化 policy，无需额外训练 value model，因此在大规模 VLA 后训练中具有较强吸引力。

具有代表性的方法通常在相同任务或可比初始条件下采样多个完整 rollout，并根据其最终结果估计组相对优势 (group-relative advantages) (Li et al. 2025; Zhang et al. 2025a)。然而，这类方法主要在完整 rollout 的层面区分 policy 表现，导致同一轨迹中不同阶段与不同动作共享统一或高度耦合的 credit signal。与之不同，我们的工作在不修改 VLA 架构、也不引入新的 value model 的前提下，重新设计了结果驱动 GRPO 中结果比较单元与时间 credit assignment 的方式。

4.3 基于结构化任务进展的时序 credit assignment

为缓解稀疏奖励或延迟奖励带来的问题，Hindsight Experience Replay 会对失败轨迹中实际达成的目标进行重标注；RUDDER 则将整条轨迹的 return 重新分配到关键时间步；此外，还有一类相关方法通过轨迹对齐或学习式 return 模型来恢复局部贡献 (Andrychowicz et al. 2017; Arjona-Medina

et al. 2019; Patil et al. 2020; Zhang et al. 2023b)。段级反馈 (segment-level feedback) 与阶段感知奖励 (stage-aware rewards) 能够提升监督信号在时间维度上的分辨率, 而局部 group-relative 方法则通过对相似状态或决策重新分组, 估计更细粒度的 advantage (Du et al. 2025; Chen et al. 2025a; Feng et al. 2026)。然而, 在连续的机器人 rollout 中, 轨迹很少会经过完全相同的视觉状态; 不同 rollout 在运动路径、阶段时长和执行速度上也常常存在显著差异, 因此, 基于原始状态的跨 rollout 分组往往并不可靠。

Reward Machines 通过高层事件与状态转移来表征任务进展; 基于 object 和 scene graph 的方法则利用结构化关系, 对连续的机器人状态与轨迹进行抽象 (Icarte et al. 2018; Furelos-Blanco et al. 2023; Sieb et al. 2020; Kumar et al. 2023; Huang et al. 2024; Zhang et al. 2023a)。不过, 这些表征主要用于任务定义、规划、奖励构造或轨迹分段, 而不是直接服务于 group-relative VLA 优化中的 advantage 估计。我们利用可验证的任务阶段, 在不同 rollout 之间对齐具有等价任务进展的动作区间, 并在以阶段为条件的分组内计算相对 advantage, 从而将轨迹级 credit assignment 重构为面向具体阶段的时序 credit assignment。

4.4 问题表述与分析

本节首先在 GRPO 框架下形式化基于结果驱动的 VLA 后训练 (post-training) 问题, 并明确说明如何将轨迹级 advantage 分配给一次 rollout 中的所有动作。随后, 我们分析这种分配方式为何会将不同的阶段推进模式映射为相同的 credit signal, 从而导致轨迹级 credit aliasing, 并进一步说明为何需要采用阶段条件化的时序 credit assignment。

4.5 问题表述

我们将 VLA 模型表述为一个参数化 policy π_θ , 它将时刻 t 的多模态状态 s_t 映射为机器人动作 a_t :

$$a_t \sim \pi_\theta(a_t \mid s_t). \quad (1)$$

其中, s_t 表示 policy 可获得的多模态输入, 包括当前视觉观测、自然语言任务指令, 以及可选的 proprioceptive 信息。对于那些在每个决策步预测多个未来控制命令的 VLA policy, 模型输出的是一个时间跨度为 H 的 action chunk:

$$\mathbf{a}_{t:t+H-1} \sim \pi_\theta(\mathbf{a}_{t:t+H-1} \mid s_t). \quad (2)$$

为简化记号, 后文统一用 a_t 表示单个动作或一个 action chunk。

强化学习后训练 (post-training) 进一步通过与目标环境交互来优化 policy。在环境交互过程中, policy 会生成一条长度为 T 的轨迹:

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T), \quad (3)$$

并在每个时间步接收一个标量反馈 r_t 。强化学习后训练的一般目标是最大化期望折扣回报:

$$\mathcal{T}_{\text{RL}}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right], \quad (4)$$

其中, $\gamma \in [0, 1]$ 为折扣因子。在本文考虑的 outcome-driven 设定中, 环境反馈主要由最终任务结果决定, 并可概括为轨迹级的 outcome reward $R(\tau)$ 。

在本文关注的 outcome-driven VLA 强化学习设定下, Group Relative Policy Optimization (GRPO) 会从相同的初始任务状态出发采样多条完整 rollout, 并根据同一 rollout 组内的结果估计相对优势。具体而言, 给定从任务分布 $\mathcal{D}$ 中采样得到的初始状态 s_0 , 旧 policy $\pi_{\theta_{\text{old}}}$ 生成一组包含 G 条动作序列的样本:

$$s_0 \sim \mathcal{D}, \{a_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | s_0), \quad (5)$$

第 i 条 rollout 表示为

$$\tau_i = (s_{i,0}, a_{i,1}, s_{i,1}, \dots, a_{i,T_i}, s_{i,T_i}), \quad (6)$$

其中, T_i 表示 rollout τ_i 中实际执行的动作数。

记第 i 条 rollout 的轨迹级 outcome reward 为 R_i 。组相对优势通过对 rollout 组内的 outcome reward 做归一化得到:

$$\hat{A}_i = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^G)}{\text{std}(\{R_j\}_{j=1}^G) + \epsilon}. \quad (7)$$

对于第 i 条 rollout 在时刻 t 生成的动作, policy 概率比定义为

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(a_{i,t} | s_{i,t})}{\pi_{\theta_{\text{old}}}(a_{i,t} | s_{i,t})}. \quad (8)$$

policy 通过如下目标函数进行优化:

$$[\text{formula}] \quad (9)$$

4.6 轨迹级 credit aliasing

GRPO 提供了一种无 value model 的组相对优化 (group-relative optimization) 方案, 其中每条 rollout 的 advantage 由其完整轨迹的结果计算得到。因此, 同一个轨迹级 advantage $\hat{A}_i$ 会在所有动作位置上被重复使用:

$$\hat{A}_{i,t} = \hat{A}_i, \quad t = 1, \dots, |a_i|. \quad (10)$$

也就是说, 完整 rollout 同时承担了两种角色: 一方面, 它是组相对 advantage 估计中的比较单位; 另一方面, 它也是 advantage 被分配时所依赖的时间单位。

长时程机器人任务通常包含多个具有不同物理语义的局部阶段, 例如到达目标物体、建立稳定抓取、搬运物体, 以及完成放置。我们将第 i 条 rollout 的阶段进展表示为

$$[\text{formula}] \quad (11)$$

其中, $z_{i,k} = 1$ 表示第 k 个任务阶段已经达成。GRPO 依据完整轨迹的结果奖励来计算组相对 advantage, 而阶段进展向量 $\mathbf{z}_i$ 并不会直接进入 advantage 估计过程。因此, 同一组中的两条 rollout 可能满足

$$[\text{formula}] \quad (12)$$

例如，一条 rollout 可能已经完成抓取和搬运，仅在最后的放置阶段失败；而另一条 rollout 可能在建立稳定抓取之前就已经失败。在二值结果奖励下，这两条 rollout 都会得到相同的失败奖励，因此，它们不同的阶段进展模式不会体现在各自的组相对 advantage 中。

当一个 rollout 组同时包含成功轨迹和失败轨迹时，所有得到相同失败奖励的 rollout 都会获得相同的负轨迹级 advantage。根据式 (10)，这一负 advantage 会被赋给该失败 rollout 中所有已经执行的动作，包括那些已经成功完成前序阶段的动作，以及后续失败阶段中的动作。我们将“把不同阶段进展模式映射到同一个轨迹级 advantage”与“将该 advantage 均匀分配给所有已执行动作”这两者的结合，称为轨迹级 credit aliasing。

这一分析表明，在长时程机器人强化学习中，credit 估计需要保留阶段进展上的差异，以及失败发生的时间位置。

5 Method

5.1 Overview

为解决轨迹级 credit aliasing 问题，我们提出了 Temporal GRPO。该方法根据任务指令与初始视觉观测，构造出一个按顺序排列、可检测的任务阶段序列，并将每条完整 rollout 与其对应的分阶段动作区间进行对齐。对于每个阶段，Temporal GRPO 仅比较那些已经完成其前置阶段并进入当前阶段的 rollout，并根据它们的阶段完成结果计算 group-relative advantage。随后，将得到的阶段 advantage 只分配给其对应的动作区间，从而在优化单一 VLA policy 的统一 GRPO 目标时，得到一个随任务进展而变化的时序 advantage。

5.2 任务条件阶段生成

为了区分一次完整 rollout 中不同的任务进展，我们根据任务指令 l 与初始视觉观测 o_0 构造一个有序的任务阶段序列：

$$M = F_{\text{stage}}(l, o_0) = (m_1, m_2, \dots, m_K), \quad (13)$$

其中， M 表示生成得到的阶段序列， m_k 表示第 k 个阶段，用于后续的 rollout 对齐与 credit assignment。该阶段序列描述了任务从初始状态推进到最终目标的主要过程，但并不将任务拆分为彼此独立训练的子 policy。每个阶段都具有明确的任务语义、有序的位置以及可检测的完成条件，因此可以作为局部结果比较与时间 credit assignment 的基本单位。

具体而言，我们首先使用一个冻结的 RynnBrain-4B 模型 (Dang et al. 2026)，在任务指令与初始场景条件下提出候选语义阶段：

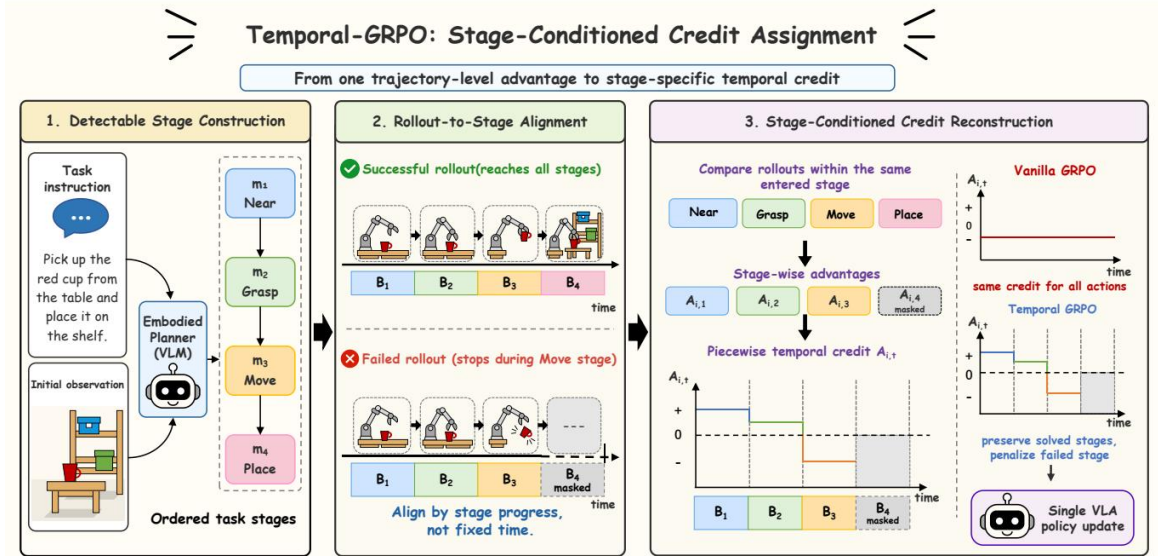


图 1: Temporal GRPO 总览：该方法将 rollouts 与可检测的任务阶段对齐，并为 VLA policy 优化重构阶段特定的时间优势。

$$\widetilde{M} = F_{\text{sem}}(l, o_0) = (\widetilde{m}_1, \widetilde{m}_2, \dots, \widetilde{m}_{\widetilde{K}}), \quad (14)$$

其中， $\widetilde{M}$ 给出了完成任务所需主要进展的语义描述。任务指令规定了目标实体、所需操作以及期望关系，而初始观测则将这些语义锚定到当前场景中的具体物体及其初始状态上。候选阶段能够从语义上刻画任务进度，但其自然语言描述可能过于抽象，或者缺乏稳定的检测边界，因此无法直接用于阶段级结果估计。

为此，Stage Compiler 会将候选语义阶段转换为可用于轨迹检测与 credit assignment 的有序 credit stages：

$$M = F_{\text{comp}}(\widetilde{M}, l, o_0) = (m_1, m_2, \dots, m_K). \quad (15)$$

这一编译过程会规范化阶段顺序及其先决依赖关系，将抽象的阶段描述转化为后续检测模块可判定的完成条件，并消除冗余或边界含糊的候选阶段。最终得到的 credit stages 服从线性的先决结构：

$$m_1 m_2 \cdots m_K, \quad (16)$$

从而只有在确认 m_{k-1} 已完成之后，才会评估阶段 m_k 。最终阶段 m_K 与原始的任务成功条件相对应，从而保证局部的阶段级 credit 与完整任务目标保持一致。Stage Compiler 的输出 schema、编译规则以及完整示例见补充材料。对于每一个任务实例，编译后的阶段序列只生成一次，并在后续所有 rollouts 中共享。

5.3 Rollout-to-Stage 对齐

编译后的 credit stage 描述的是任务层面的进展；而 policy 优化则需要实际 rollout 中识别与各个 stage 对应的动作。给定 stage 序列 $M = (m_1, \dots, m_K)$ 以及第 i 个完整 rollout，我们根据各 stage 的完成条件确定其在 rollout 中达成的位置，并将动作序列与相应的 stage 区间对齐。该对齐由

任务状态转移决定。在仿真实验中，stage 谓词仅在 post-training 阶段基于 simulator 的 privileged state 进行判定；在评估时，既不使用该 detector，也不使用 privileged state。

relation detector 会沿着 rollout 评估各个完成条件，并且只有在确认 stage m_{k-1} 已完成后，才开始验证 stage m_k 。对于每个已完成的 stage，我们记录其完成条件首次被稳定满足的时刻：

$$T_{i,k} = \min \{t \mid m_k \text{ is stably completed in rollout } i\}. \quad (17)$$

稳定性约束用于避免将瞬时接触、短暂的空间重合或视觉抖动误判为 stage 完成。relation 条件、稳定性判据以及 detector 的具体实现见补充材料。

对于成功完成的 stage m_k ，其对应的动作区间定义为

$$B_{i,k} = (T_{i,k-1}, T_{i,k}], \quad T_{i,0} = 0. \quad (17)$$

该区间包含推动 rollout 从已完成的前置 stage m_{k-1} 进入 stage m_k 的动作。若 rollout i 已完成 m_{k-1} ，但在终止前未能完成 m_k ，则将其剩余的动作后缀分配给 stage m_k 的失败区间：

$$B_{i,k} = (T_{i,k-1}, T_i], \quad (18)$$

其中， T_i 表示 rollout 的终止时刻。在这种情况下， m_k 被视为当前失败的 stage，而后续 stage 则被视为尚未到达；它们既不会被分配有效区间，也不会被纳入后续 stage 的比较中。因此，每个 rollout 都会与一串按 stage 划分的动作区间对齐，这些区间可能对应成功，也可能对应失败；它们为后续的 stage-outcome 构造和 group-relative advantage 估计提供了时间边界。

5.4 阶段条件优势重建与策略优化

对于阶段 m_k ，只有已经完成其前置阶段的 rollout，才被视为进入当前阶段，并参与对应的组内相对比较。我们将阶段参与变量定义为

$$V_{i,k} = [\text{formula}] \quad (19)$$

当 $V_{i,k} = 0$ 时，表示 rollout i 尚未满足阶段 m_k 的前置条件，因此应将其排除，而不是将其视为该阶段上的失败样本。

对于每个满足 $V_{i,k} = 1$ 的 rollout，我们定义其二值阶段结果为

$$R_{i,k} = [\text{formula}] \quad (20)$$

最终阶段 m_K 的结果与原始任务成功信号完全一致。阶段 m_k 的结果统计量只在其有效 rollout 上计算：

$$\mu_k = \frac{\sum_{i=1}^G V_{i,k} R_{i,k}}{\sum_{i=1}^G V_{i,k}}, \quad \sigma_k = \sqrt{\frac{\sum_{i=1}^G V_{i,k} (R_{i,k} - \mu_k)^2}{\sum_{i=1}^G V_{i,k}}}. \quad (21)$$

在此基础上，阶段条件的组相对优势定义为

$$\hat{A}_{i,k} = \frac{R_{i,k} - \mu_k}{\sigma_k + \epsilon}, \quad V_{i,k} = 1. \quad (22)$$

如果某个阶段没有任何有效 rollout，或者其所有有效 rollout 的结果完全相同，那么该阶段就无法提供相对排序信号，因此在当前策略更新中会被跳过。

我们仅将每个阶段的优势分配给产生对应阶段结果的动作区间，从而基于原本轨迹级别的信号，重建出一个随时间变化的优势：

$$\hat{A}_{i,t} = \sum_{k=1}^K \mathbb{I}[t \in B_{i,k}] \hat{A}_{i,k}. \quad (23)$$

由于各阶段区间是有序且互不重叠的，因此每个动作至多继承一个有效的阶段优势。对于预测 action chunk 的 VLA policy，每个 action chunk 及其内部的 action token，都继承其所属阶段区间的优势。

最后，我们用阶段条件的时序优势 $\hat{A}_{i,t}$ 替换原先统一广播到整条轨迹的轨迹级优势 $\hat{A}_i$ ：

$$[\text{formula}] \quad (24)$$

该目标函数保留了完整 rollout 采样、policy 似然比以及带裁剪的 GRPO 优化机制，同时将 credit comparison 与 credit assignment 从整条完整轨迹细化到阶段特定的动作区间。所有阶段区间都在同一个训练 batch 中、通过统一的目标函数联合优化，以更新同一个 VLA policy，而不是针对不同阶段分别执行彼此独立的强化学习。

5.5 实验

我们围绕三个问题开展评估：其一，Temporal GRPO 是否能够提升最终任务成功率；其二，在固定交互预算下，它是否具有更高的学习效率；其三，它的 policy 更新是否主要集中在区分成功与失败 rollout 的关键阶段。任务层面的性能与学习曲线评测采用 RoboTwin 2.0，而受控的 credit assignment 分析与消融实验则采用 LIBERO-Long。

6 Experimental Setup

Benchmarks. 我们在 RoboTwin 2.0 (Chen et al. 2025b) 上评估整体任务性能以及强化学习的样本效率。进一步地，我们在具有清晰阶段依赖关系的 LIBERO-Long (Liu et al. 2023) 任务上开展受控的 credit assignment 分析与消融实验。RoboTwin 2.0 支持在不同任务时域长度下进行评测，而 LIBERO-Long 则便于分析共享前置阶段，以及 rollout 结果首次发生分歧的阶段。四个标准 LIBERO suite 上的完整结果见补充材料。

Baselines. 我们将所提方法与具有代表性的 VLA baseline: π_0 (Black et al. 2026) 和 RDT-1B (Liu et al. 2025) 进行比较，同时对比 VLA-RL 方法 SimpleVLA-RL (Li et al. 2025) 与 TGRPO (Chen et al. 2025d)。在受控强化学习对比中，SimpleVLA-RL、TGRPO、Trajectory-GRPO、Stage-Reward GRPO 和 Temporal GRPO 均从同一个任务特定的 OpenVLA-OFT SFT checkpoint 初始化。Trajectory-GRPO 将最终结果对应的 advantage 分配给完整动作序列；Stage-Reward GRPO 则将检测到的相同阶段进展转换为标量奖励，但仍保留基于整条 trajectory 的 advantage 分配方式。所有受控强化学习方法均使用相同的训练场景、rollout budget、环境交互 budget、policy update budget 以及评测协议。

表 1: RoboTwin 2.0 上不同任务时域长度的任务成功率 (%)。所有 RL 方法均从同一组公开发布的任务特定 OpenVLA-OFT SFT checkpoints (Li et al. 2025) 初始化, 并在完全一致的 budget 下训练。RL 结果报告为 3 个训练随机种子的均值 $\pm$ 标准差。

Method	Short Horizon	Medium Horizon	Long & Extra-Long	Macro Avg.
π_0	45.5	58.8	43.3	49.2
RDT-1B	24.5	47.8	27.8	33.3
OpenVLA-OFT (SFT)	21.3	47.1	46.5	38.3
Trajectory-GRPO	37.8 ± 1.8	52.6 ± 1.6	48.7 ± 1.9	46.4 ± 1.3
TGRPO	43.9 ± 1.6	58.4 ± 1.5	54.1 ± 1.7	52.1 ± 1.1
Stage-Reward GRPO	52.7 ± 1.4	64.2 ± 1.3	60.8 ± 1.5	59.2 ± 1.0
SimpleVLA-RL	64.9 ± 1.2	72.5 ± 1.0	69.0 ± 1.3	68.8 ± 0.9
Temporal GRPO	73.2 ± 0.9	79.0 ± 0.8	75.2 ± 1.1	76.0 ± 0.7

Implementation Details. 对于每个 RoboTwin 2.0 任务,我们都使用公开发布的任务特定 OpenVLA-OFT SFT checkpoint, 作为所有受控强化学习方法共享的 warm-start。Trajectory-GRPO、Stage-Reward GRPO 和 Temporal GRPO 采用完全相同的探索与优化超参数, 以隔离 credit-assignment 规则本身的影响。SimpleVLA-RL 和 TGRPO 则在匹配的 rollout、环境交互和 policy update budget 下, 使用其各自推荐的配置。

对于每一种任务 specification, 在强化学习开始前, 冻结的 RynnBrain-4B 模型会一次性生成候选语义阶段; 这些阶段随后会在该任务的所有 rollouts 中复用。Stage Compiler 将候选阶段转换为 credit stages, 并为其附加前置依赖关系与可检测的完成条件; 之后, relation detector 识别这些阶段的首次完成时刻, 并构造相应的动作区间。特权 simulator 状态以及与阶段相关的模块仅用于强化学习后训练阶段的 credit 构造, 在 policy 评测时不会调用。完整的任务阶段、relation predicates、训练超参数和评测配置均见补充材料。

6.1 实验设置 (第二部分)

评估指标。 主要评估指标为任务成功率(success rate)。对于受控强化学习实验,结果以三个独立训练随机种子下的均值和标准差进行报告; 对每个 checkpoint, 均在每个任务上使用固定数量的留出 rollouts 进行评估。样本效率通过固定环境交互预算下的任务成功率来衡量。对于受控 credit-assignment 分析, 我们还额外报告 policy 更新前后各阶段条件完成概率 (stage-conditional completion probabilities) 的变化。

6.2 Overall Performance and Sample Efficiency

表 1 汇报了在不同 RoboTwin 2.0 任务时域下的任务成功率。对于强化学习方法, 结果表示为 3 个独立训练随机种子上的均值和标准差。Temporal GRPO 在每个任务时域分组中都取得了最高成功率。其宏平均成功率达到 75.8 ± 0.7 ., 是所有对比方法中的最佳结果。在 Long 和 Extra-Long 任务上的提升, 支持了将阶段条件信用分配用于具有序列阶段依赖的长时域 manipulation。

图 2 展示了在全部 RoboTwin 2.0 任务以及 Long 与 Extra-Long 子集上, 任务成功率随强化学习训练步数变化的曲线。所有方法在每个训练步使用相同数量的 rollout, 因此在相同训练步下的比较对应于相同的环境交互预算。整个后训练过程中, Temporal GRPO 始终保持高于 Trajectory-GRPO 的任务成功率。这种效率优势在 Long 与 Extra-Long 子集上依然十分明显, 因为这类任务的结果反馈更延迟, 时间信用分配也更具挑战。

这些结果表明, 阶段条件信用分配能够在相同 rollout 预算下提取更有效的训练信号, 从而带来更快的性能提升和更高的最终任务成功率。

6.3 Controlled Credit Assignment

仅看整体任务成功与否, 无法判断一次 policy 更新究竟改善了哪些任务阶段, 或削弱了哪些阶段。为了直接考察 credit 被分配到何处, 我们从自然采样得到的 rollouts 中选取匹配的分组。对于每个分组, 其内部轨迹都成功完成了相同的前置阶段, 并且首次在阶段完成结果上出现分歧的阶段记为 m_d 。对于阶段 m_k , 我们通过进入该阶段的 rollouts 的完成概率变化来度量更新效果:

$$[\text{formula}] \quad (25)$$

其中, $V_k(\tau)$ 和 $R_k(\tau)$ 分别表示 rollout τ 是否进入阶段 m_k 以及是否完成该阶段。 Δp_k 为正表示该阶段的完成情况得到提升, 为负则表示该阶段性能退化; 其单位为百分点。

图 3 按首次产生差异的阶段 m_d 对不同的 LIBERO-Long 任务进行对齐, 并在相对阶段位置上比较 Δp_k 。Trajectory-GRPO 在共享的前置阶段上产生了明显的负向变化, 这表明其基于整条轨迹的负 advantage 会传播到那些原本已经成功的动作区间。相比之下, Temporal GRPO 能够将前置阶段上的变化保持在接近零的水平, 并在 m_d 处带来最大的正向提升。这说明它能够保留已经习得的前序行为, 并将更新集中作用于导致 rollout 差异的关键阶段。

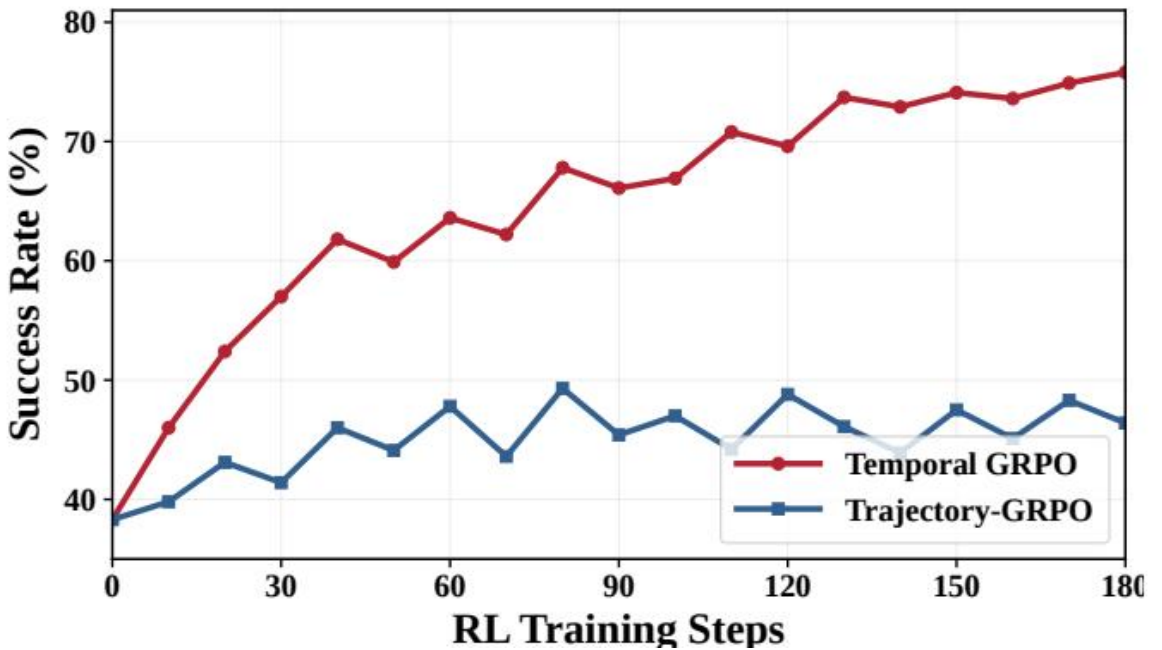


图 2: RoboTwin 2.0 上 Temporal GRPO 与 Trajectory-GRPO 的样本效率比较: (a) 全部 RoboTwin 2.0 任务。

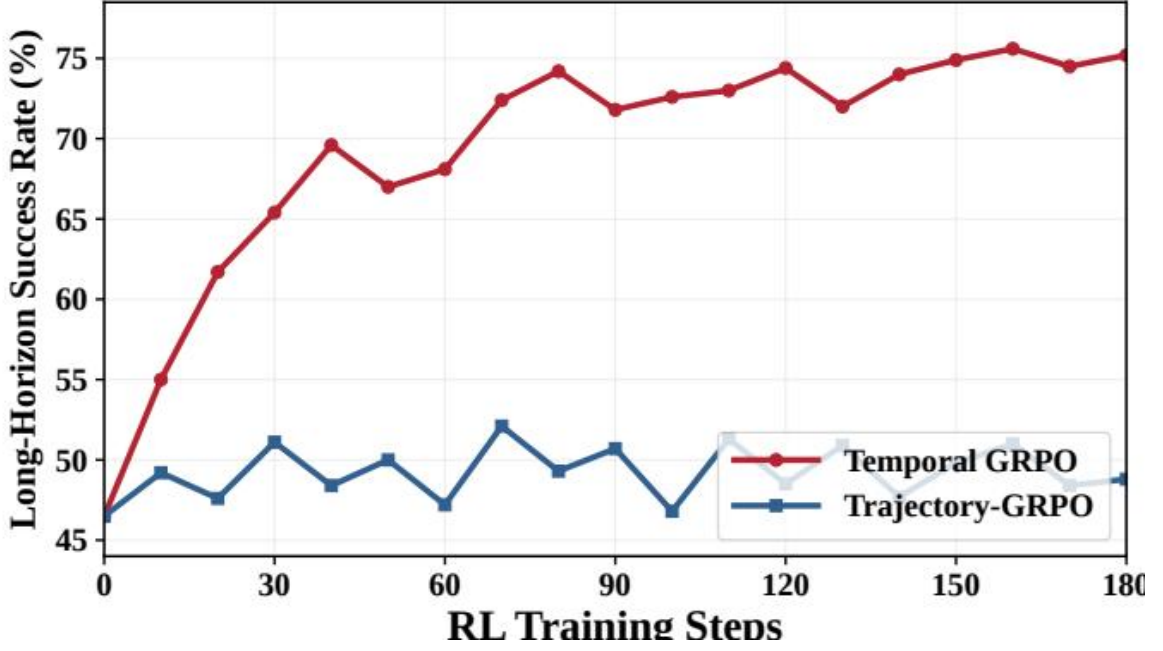


图 3: RoboTwin 2.0 上 Temporal GRPO 与 Trajectory-GRPO 的样本效率比较: (b) Long 与 Extra-Long 任务。曲线给出了在三组独立训练随机种子下的平均任务成功率 (success rate)。

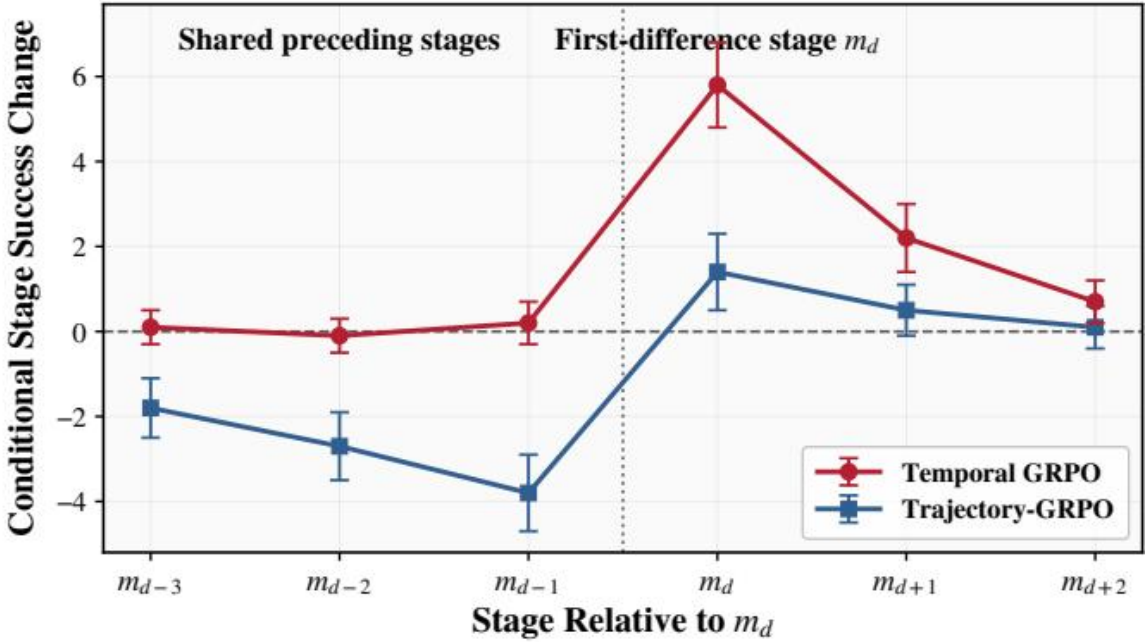


图 4: LIBERO-Long 上受控的逐阶段 credit assignment 分析。任务按照首次产生差异的阶段 m_d 对齐。图中的标记点与误差条分别表示以百分点计的平均 Δp_k 及其一个标准差。

6.4 消融实验

我们在 LIBERO-Long 上评估了所有消融变体, 使用相同的 policy 初始化与训练预算, 并报告 3 个独立随机种子下成功率 (success rate) 的均值和标准差。若不使用 entered-stage gating, 则尚未进入当前 stage 的 rollout 会被视为该 stage 失败, 从而导致性能明显下降。移除 same-stage grouping 会带来最大的组件级性能退化, 这表明来自不同 stage 的动作区间并不能构成可靠的共享相对比较

组。不使用 Stage Compiler 时出现的性能下降说明, 按序排列的前置条件以及可检测的完成条件, 能够提升 rollout 到 stage 对齐的可靠性。

Stage-Reward GRPO 始终优于 Trajectory-GRPO, 这验证了 stage 进度能够提供额外且有效的监督信号。Temporal GRPO 则进一步取得了最优性能, 说明基于 stage 条件的比较, 以及将 advantage 分配给对应动作区间, 对于获得完整的性能提升仍然是关键。

表 2: 在相同训练预算下, LIBERO-Long 上的消融结果。结果以 3 个随机种子下的均值 $\pm$ 标准差报告。

Variant	Success Rate (%) $\uparrow$
Temporal GRPO	99.1 $\pm$ 0.4
w/o Stage Compiler	96.8 $\pm$ 0.7
Stage-Reward GRPO	94.7 $\pm$ 0.9
w/o entered-stage gating	92.5 $\pm$ 1.1
w/o same-stage grouping	90.6 $\pm$ 1.3
Trajectory-GRPO	88.4 $\pm$ 1.5

7 Conclusion

本文提出 Temporal GRPO, 用于解决以结果为驱动的 VLA 后训练中, 轨迹级 credit aliasing 问题。该方法构造可检测且有序的任务阶段, 在共享先决进度的条件下, 对进入同一阶段的 rollouts 进行比较, 并将阶段相对 advantage 分配到对应的动作区间。RoboTwin 2.0 上的实验表明, 该方法在不同任务时域下都能提升任务成功率 (success rate) 和样本效率; 同时, LIBERO-Long 上受控的分析实验与消融实验也支持了所提出的阶段局部化 credit 分配机制。

目前, Temporal GRPO 仍依赖可靠的阶段谓词以及预先定义的线性阶段顺序。当阶段边界存在歧义, 或任务进展包含分支、重复阶段、以及通过回退到更早阶段进行恢复时, 这一设定可能会限制阶段对齐与 credit 分配的效果。未来的工作将探索具备不确定性感知的阶段检测, 以及动态阶段图, 从而将时序 credit 分配扩展到更复杂的机器人操纵过程。

8 参考文献

参考文献条目较多, 此处保留原论文英文文献列表, 请参阅原文 PDF 的 References 一节。